Agentic Memory Lifecycle Phase	Type of Data	Life of Data	Type of Data Structure	Memory Type	Operation	Producer → Consumer	Access Frequency	Routing Rule
System Bootstrap	Orchestrator prompt Agent prompts Task type rules (A-E) Agent capability profiles Embedding model FAISS indexes from disk	Permanent (never dies) Exception: FAISS loaded from disk (persists across runs)	Strings (prompts) Dicts (profiles) FAISS IndexFlatIP SentenceTransformer	PM (Procedural) ENT (Entity) SM (Semantic) NONE (infra)	WRITE (init) LOAD	Developer → PM store Developer → Entity store Disk → FAISS RAM Disk → Model RAM	Once at startup (never again)	Static rules → PM Facts about things → ENT Meaning vectors → SM Infrastructure → not memory
Task Ingestion	Task description Username Task date Task pattern classification Empty step history Empty shared context	All die when task ends (volatile)	Strings (task, user) String YYYY-MM-DD String (pattern enum) Empty list [] Empty dict {}	WM (Working) for ALL items	WRITE for ALL items	User/Config → WM store Classifier → WM store	Once per task (at task start)	All data changes between tasks and dies when task ends → Working Memory
Memory Retrieval (route_query)	Orch prompt (from PM) Task context (from WM) Cached bundle (from STM) Past experiences (from EM) Meaning vectors (from SM) User profile (from ENT)	Varies: PM: permanent WM: per task STM: session EM/SM: on disk ENT: session	Strings (prompts) Dicts (context) Dicts (bundles) FullTaskMemory objs float32[384] vectors Dicts (profiles)	PM (Procedural) WM (Working) STM (Short-Term) EM (Episodic) SM (Semantic) ENT (Entity)	READ for ALL items SKIP entity if pattern says not needed	PM → Orch LLM WM → Orch LLM STM → MemMgr FAISS → MemMgr ENT → Orch LLM	PM+WM: every call STM: once/task EM+SM: on miss ENT: on miss + if pattern needs	PM+WM: ALWAYS read STM: ALWAYS check first If STM HIT → skip EM,SM,ENT If STM MISS → check pattern to decide which to read
Orchestrator Inference	Assembled system prompt Assembled user prompt API account used LLM call latency Rate limit events	All die after this single LLM call (most volatile)	Strings (prompts) String (account ID) Float (seconds) Boolean (429)	NONE (consumed by LLM, not stored in memory system)	CONSUME LOG	PM+WM+EM+ENT+STM → LLM API LLM API → terminal log	Every orchestrator LLM call (3-8x per task)	Prompts assembled from multiple memory types then consumed by LLM. Not stored anywhere.
Orchestrator Output (delegation)	Thought field Agent assignment Subtask instruction Plan-driven agent detection Loop detection result Forced finish (if loop)	All die when task ends Exception: plan-driven detection triggers EPISODIC read	Strings (thought, agent, subtask) Set of strings (detected agents) Boolean (loop) Dict (forced finish)	WM (Working) for ALL items Triggered: EM READ via plan-driven loading	WRITE to WM READ+COMPARE (loop detection) Trigger EM READ	Orch LLM → WM Thought parser → triggers EM read WM history → loop detector	Once per orch step (3-8x per task) Plan-driven: once per task	Current reasoning → WM Agent detection → trigger batch EM loading Loop → force finish All volatile, per-task data
Agent Inference	Agent system prompt Agent episodic memories Subtask from orch Agent inner history Agent repeat detection	Prompt: permanent Memories: on disk Subtask: per task Inner history: per agent turn (shortest)	Strings (prompt) SubtaskMemory list String (subtask) List of {action, obs} Boolean	PM (Procedural) EM (Episodic) WM (Working)	PM: READ EM: READ (pre-loaded) WM: READ Inner: READ+WRITE Repeat: COMPARE	PM store → Agent LLM Pre-loaded dict → Agent LLM WM → Agent LLM Agent loop → itself	PM: every agent call EM: once/agent call WM: once/agent call Inner: max 2x Repeat: every step	PM: each agent has OWN procedural rules EM: pre-loaded in Phase 5 (no new FAISS query) WM: instruction from orch
Agent Output (action)	Agent action JSON Calendar event file (.ics) Email file (.eml) Answer file (answer.txt)	Action: per task Files: persist on disk permanently	Dict {app, action, user, summary, time} .ics (VEVENT) .eml (RFC 2822) .txt (plain text)	WM (Working) for action JSON EXT (External) for all files	WM: WRITE EXT: WRITE (file creation)	Agent LLM → execute_fs_action → File system Action consumed by executor	Once per agent call Files: once per create/send action	Action JSON → WM (consumed by executor) Files → EXTERNAL (environment, NOT memory. Like a carpenter's table, not the carpenter's memory)
Environment Observation	Action result string Email list data (structured) Calendar list data Step record → history Cross-agent shared context	All die when task ends (volatile)	Strings "OBSERVATION:..." (100-1500 chars) Dicts appended to step_history list Dict {agent: {obs}}	WM (Working) for ALL items	WRITE for ALL items	File system → WM store → Orchestrator reads next step Agent obs → shared_context → other agents see	Once per agent step (written once, read every subsequent step)	All are current task results that die when task ends. Shared across agents via WM. Orch reads history every step. Email agent sees calendar result.
Memory Storage (on_task_ complete)	Success/failure flag Task pattern bundle Step signature Task embedding User profile update Working memory CLEARED Evaluation metrics	Flag: per task Bundle: session Signature: session Embedding: session Entity: session WM: DIES NOW Metrics: per task	Boolean Dict {plan, mems} String "a:x → b:y" float32[384] Dict (merged) All fields → None Dict {passed, total}	WM: flag STM: bundle+sig+emb ENT: profile WM: CLEAR NONE: metrics	WM: READ (flag) STM: WRITE ×3 ENT: WRITE WM: CLEAR Metrics: LOG	Evaluation → MemMgr Execution → STM cache Execution → Entity MemMgr → WM (clear) Evaluator → JSON log	Once per task end STM: success only ENT: success + new info only WM: always cleared	Success → write STM + ENT Failure → only clear WM Bundle shortcuts future → STM User facts persist → ENT Task context dies → WM clear Metrics → not stored
Memory Consolidation (after training)	Distilled FullTaskMemory Subtask memories (per agent) FAISS embedding vectors Consolidated rules (learned) Pre-populated STM patterns	EM: persists on disk SM: persists on disk PM rules: session STM: session	FullTaskMemory objects SubtaskMemory objects float32[384] vectors List of rule dicts Same as task bundles	EM (Episodic) SM (Semantic) PM (Procedural) STM (Short-Term)	WRITE for ALL items EM: distill SM: embed PM: consolidate STM: pre-fill	LLM curator → FAISS + JSON Embed model → FAISS Consolidation → PM learned_rules Training → STM cache	Once after ALL training completes (batch operation) Not per-task	Specific experience → EM Meaning vector → SM 10+ similar episodes merged into rule → PM Successful patterns cached for testing → STM